COORDENADAS DE MANO EN

	A	B
1	8	1
2	7	2
3	6	5
4	6	8
5	6	11
6	5	14
7	4	17
8	3	20
9	3	22
10	5	22
11	6	20
12	7	18
13	8	16
14	9	15
15	9	17
16	8	21
17	7	24
18	7	27
19	9	28
20	10	26
21	11	23
22	11	20
23	12	17
24	12	21
25	12	25
26	12	28
27	13	30
28	15	29
29	15	25
30	15	21
31	16	17
32	17	18
33	18	21
34	19	25
35	20	27
36	22	26
37	22	23
38	21	20
39	20	16
40	20	13
41	21	10
42	23	13
43	25	15
44	27	16
45	28	14
46	27	11
47	24	8
48	23	5
49	20	1

INTERPOLACION PARAMÉTRICA:

LAGRANGE

NEWTON

MATRICIAL

metodos_interpolacion.m | spline_cubico.m

```
% INTERPOLACIÓN PARAMÉTRICA (Mano - 49 Puntos)
% Métodos: Lagrange, Newton y Matricial
clear; clc;

% ---------------------------------------------------------------------
% 1. CARGA DE DATOS (Selección de Excel)
% ---------------------------------------------------------------------
fprintf('Seleccione el archivo de Excel con los datos...\n');
[archivo, ruta] = uigetfile({'*.xlsx;*.xls', 'Archivos de Excel'}, 'Seleccione el Excel');

if isequal(archivo, 0)
    disp('Operación cancelada por el usuario.');
    return;
end

datos = readmatrix(fullfile(ruta, archivo));
x = datos(:, 1)'; % Coordenadas X
y = datos(:, 2)'; % Coordenadas Y
n = length(x);

% Parámetros de interpolación
T = 1:n; % Vector de tiempo paramétrico (1 a 49)
tt = linspace(1, n, 500); % Puntos a evaluar

fprintf('\nArchivo cargado exitosamente: %s\n', archivo);
fprintf('Procesando %d puntos...\n', n);

% ---------------------------------------------------------------------
% 2. MÉTODO 1: POLINOMIO DE LAGRANGE
% ---------------------------------------------------------------------
disp('Calculando Lagrange...');
x_lagrange = zeros(1, length(tt));
y_lagrange = zeros(1, length(tt));

for k = 1:length(tt)
    sum_x = 0; sum_y = 0;
    for i = 1:n
        L = 1;
        for j = 1:n
            if i ~= j
                L = L * (tt(k) - T(j)) / (T(i) - T(j));
            end
        end
        sum_x = sum_x + L * x(i);
        sum_y = sum_y + L * y(i);
    end
    x_lagrange(k) = sum_x;
    y_lagrange(k) = sum_y;
end
```

```matlab
% ------------------------------------------------------------------------
% 3. MÉTODO 2: POLINOMIO DE NEWTON (Diferencias Divididas)
% ------------------------------------------------------------------------
disp('Calculando Newton...');
Dx = zeros(n, n); Dx(:,1) = x';
Dy = zeros(n, n); Dy(:,1) = y';

% Llenado de matrices de diferencias divididas
for j = 2:n
    for i = 1:n-j+1
        Dx(i,j) = (Dx(i+1,j-1) - Dx(i,j-1)) / (T(i+j-1) - T(i));
        Dy(i,j) = (Dy(i+1,j-1) - Dy(i,j-1)) / (T(i+j-1) - T(i));
    end
end

coef_newton_x = Dx(1,:);
coef_newton_y = Dy(1,:);

x_newton = zeros(1, length(tt));
y_newton = zeros(1, length(tt));

% Evaluación del polinomio
for k = 1:length(tt)
    x_newton(k) = coef_newton_x(1);
    y_newton(k) = coef_newton_y(1);
    prod_term = 1;
    for i = 2:n
        prod_term = prod_term * (tt(k) - T(i-1));
        x_newton(k) = x_newton(k) + coef_newton_x(i) * prod_term;
        y_newton(k) = y_newton(k) + coef_newton_y(i) * prod_term;
    end
end
% ------------------------------------------------------------------------
% 4. MÉTODO 3: POLINOMIO MATRICIAL (Matriz de Vandermonde)
% ------------------------------------------------------------------------
disp('Calculando Matricial (Vandermonde)...');

% Construcción de la Matriz de Vandermonde manual
V = zeros(n, n);
for i = 1:n
    for j = 1:n
        V(i, j) = T(i)^(j-1);
    end
end

% Desactivar advertencia de matriz mal condicionada temporalmente (explicación abajo)
warning('off', 'MATLAB:nearlySingularMatrix');
warning('off', 'MATLAB:illConditionedMatrix');

% Resolución de los sistemas de ecuaciones lineales (V * c = x)
coef_mat_x = V \ x';
coef_mat_y = V \ y';

warning('on', 'all'); % Reactivar advertencias

x_matricial = zeros(1, length(tt));
y_matricial = zeros(1, length(tt));

% Evaluación del polinomio matricial
for k = 1:length(tt)
    for j = 1:n
        x_matricial(k) = x_matricial(k) + coef_mat_x(j) * (tt(k)^(j-1));
        y_matricial(k) = y_matricial(k) + coef_mat_y(j) * (tt(k)^(j-1));
    end
end
```

```
% ------------------------------------------------------------------------
% 5. REPORTE FINAL
% ------------------------------------------------------------------------
fprintf('\n==========================================');
fprintf('\n       ALGORITMOS COMPLETADOS            ');
fprintf('\n==========================================');
fprintf('\n Matrices resultantes guardadas en memoria:');
fprintf('\n - x_lagrange,  y_lagrange');
fprintf('\n - x_newton,    y_newton');
fprintf('\n - x_matricial, y_matricial');
fprintf('\n==========================================\n');
```

RESULTADOS

Command Window

```
Seleccione el archivo de Excel con los datos...

Archivo cargado exitosamente: coordenadasmano.xlsx
Procesando 49 puntos...
Calculando Lagrange...
Calculando Newton...
Calculando Matricial (Vandermonde)...

==========================================
       ALGORITMOS COMPLETADOS
==========================================
 Matrices resultantes guardadas en memoria:
 - x_lagrange,  y_lagrange
 - x_newton,    y_newton
 - x_matricial, y_matricial
==========================================
```

SPLINE CÚBICO

```
clear; clc; close all;

% 1. SELECCIÓN DEL ARCHIVO
fprintf('Seleccione el archivo de Excel con los 49 puntos de la mano...\n');
[archivo, ruta] = uigetfile({'*.xlsx;*.xls', 'Archivos de Excel (*.xlsx, *.xls)'}, 'Seleccione el archivo de datos');

if isequal(archivo, 0)
    disp('Usuario canceló la selección.');
    return;
else
    fullPath = fullfile(ruta, archivo);
    datos = readmatrix(fullPath);
    x = datos(:, 1)';
    y = datos(:, 2)';
end

% 2. CONFIGURACIÓN
n = length(x);
T = 1:n;                % Parámetro de tiempo (puntos de control)
tt = linspace(1, n, 1000); % 1000 puntos para una suavidad total

% 3. MÉTODO DE SPLINES CÚBICOS (La solución definitiva)
% Calculamos el spline para X y para Y por separado respecto al tiempo T
x_spline = spline(T, x, tt);
y_spline = spline(T, y, tt);

% --- REPORTE EN CONSOLA ---
fprintf('\n========================================');
fprintf('\n       DIBUJO COMPLETO DE LA MANO        ');
fprintf('\n========================================');
fprintf('\n Método: Splines Cúbicos Paramétricos');
fprintf('\n Puntos procesados: %d', n);
fprintf('\n Estado: Curva suavizada y completa');
fprintf('\n========================================\n');

% 4. GRÁFICA
figure('Color', 'w', 'Name', 'Mano Completa - Spline Cúbico');
hold on;

% Dibujar la curva del Spline (Línea continua y suave)
plot(x_spline, y_spline, 'b-', 'LineWidth', 2.5, 'DisplayName', 'Trazo de la Mano');

% Dibujar los puntos originales del Excel
plot(x, y, 'ro', 'MarkerFaceColor', 'r', 'MarkerSize', 5, 'DisplayName', 'Puntos Originales');
```

```
% Configuración de vista
grid on;
axis equal; % CRUCIAL: Esto evita que la mano se vea gorda o flaca
title(['Dibujo de Mano Completa - Splines desde: ', archivo]);
xlabel('Coordenada X');
ylabel('Coordenada Y');
legend('Location', 'best');

% Añadir etiquetas numeradas a los puntos (opcional, para verificar orden)
% for i = 1:n
%     text(x(i), y(i), num2str(i), 'FontSize', 8, 'VerticalAlignment', 'bottom');
% end

hold off;
fprintf('\n¡Proceso terminado! Ahora deberías ver la mano completa y fluida.\n');
```

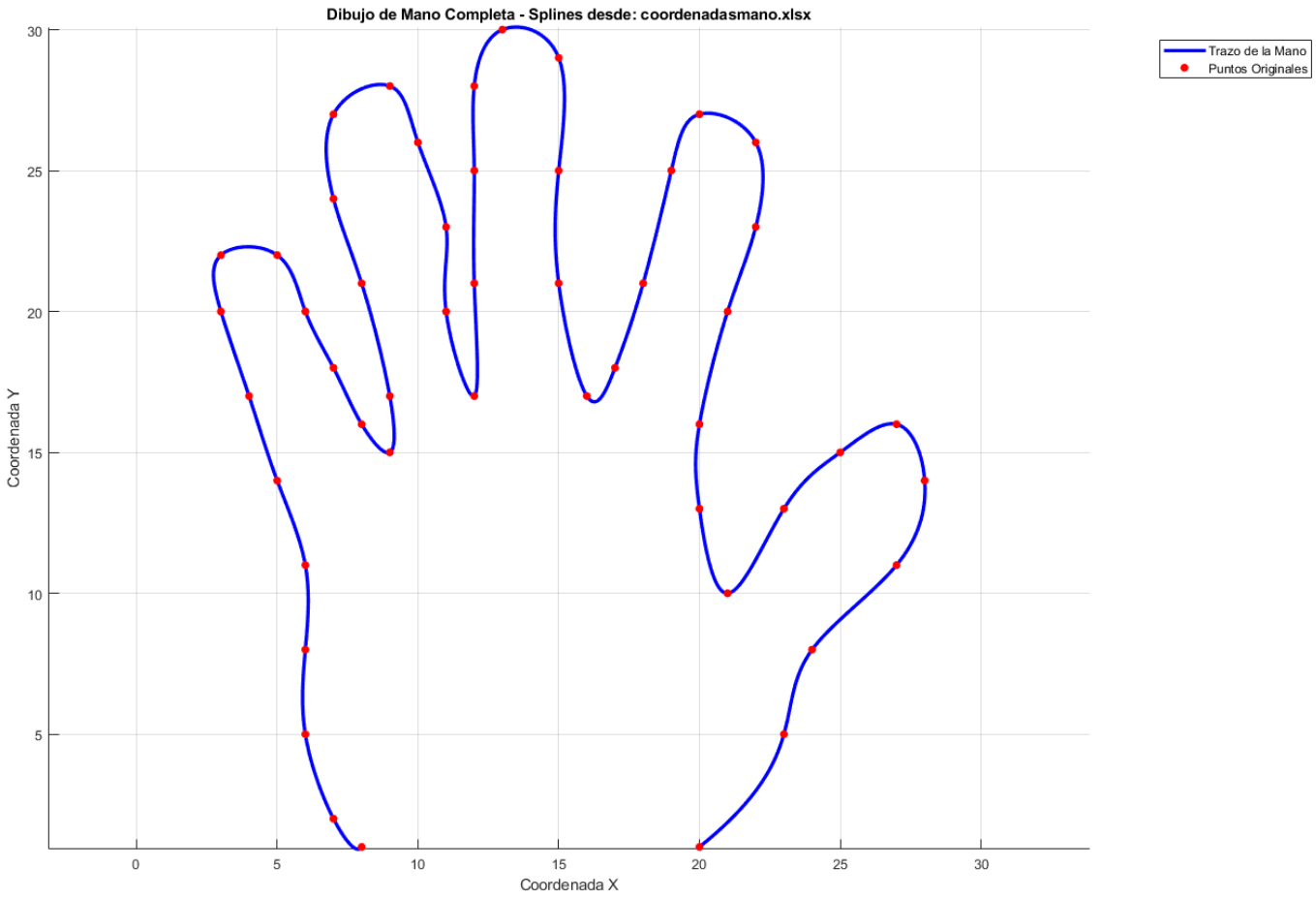
RESULTADO

```
Command Window
Seleccione el archivo de Excel con los 49 puntos de la mano...

==========================================
   DIBUJO COMPLETO DE LA MANO DE MARTIN
==========================================
 Método: Splines Cúbicos Paramétricos
 Puntos procesados: 49
 Estado: Curva suavizada y completa
==========================================

¡Proceso terminado! Ahora deberías ver la mano completa y fluida.
fx >>
```

GRÁFICA DE LA MANO DE MARTIN



PREGUNTAS:

1. ¿Por qué el método de Lagrange, Newton y matricial no responden bien?

Fallan porque el contorno de la mano tiene coordenadas X que se repiten (un mismo X tiene varios valores de Y). Al no cumplir con la condición de una función matemática $y = f(x)$, pasa a ser una simple relación matemática, haciendo que las fórmulas estándar colapsen. A esto se suma el Fenómeno de Runge: al intentar forzar un polinomio gigante de grado 48 para unir todos los puntos, la curva se vuelve muy inestable y hace oscilaciones salvajes. Además, en el método matricial, tener que elevar números a potencias tan altas hace que la computadora pierda precisión con los decimales.

2. ¿Por qué el método de splines cúbicos lo hace bien?

Porque no intenta hacer una sola función para todo, sino que divide el trazado en pequeños segmentos de grado 3. Al aplicarlo de forma paramétrica, permite dibujar formas cerradas (como los dedos) sin importar que las X se repitan. Como el grado del polinomio es bajo, evita las oscilaciones extremas, y su algoritmo asegura que las uniones entre los puntos sean perfectamente suaves y continuas.